

409. Longest Palindrome Easy

Acceptance: 55.8% | [LeetCode](#)

Given a string `s` which consists of lowercase or uppercase letters, return the length of the **longest palindrome** that can be built with those letters.

Letters are **case sensitive**, for example, "Aa" is not considered a palindrome.

Example 1:

```
**Input:** s = "abcccd"
**Output:** 7
**Explanation:** One longest palindrome that can be
built is "dccacd", whose length is 7.
```

Example 2:

```
**Input:** s = "a"
**Output:** 1
**Explanation:** The longest palindrome that can be
built is "a", whose length is 1.
```

Constraints:

- `1 <= s.length <= 2000`
- `s` consists of lowercase **and/or** uppercase English letters only.

Claude Opus Solution

Longest Palindrome

1. Intuition

The key insight comes from understanding **what makes a palindrome**:

A palindrome reads the same forwards and backwards. This means: - Characters must appear in **symmetric pairs** around the center (e.g., `abcba` → `a..a`, `b.b`). - At most **one character** can be in the exact center (the odd one out), which doesn't need a partner.

So the question reduces to: **how many characters can I pair up?**

- If a character appears an **even** number of times, all of them can be used (perfectly paired).
- If a character appears an **odd** number of times, we use the largest even portion (`count - 1`), reserving the leftover single.
- If any character had an odd count, we can place exactly **one** of those leftovers in the center → add `+1`.

Pattern used: Frequency counting (Hash Table / Counter) + parity (even/odd) reasoning.

2. Approach (Pseudocode)

1. Count the frequency of every character in `s`.
2. Initialize `length = 0` and `has_odd = False`.
3. For each character's count:
 - Add the largest even number `<= count` to `length`:
`length += count - (count % 2)`
 - If `count` is odd, mark `has_odd = True`.
4. If `has_odd` is `True`, we can add one center character → `length += 1`.
5. Return `length`.

3. Python Solution

```
from collections import Counter

class Solution:
    def longestPalindrome(self, s: str) -> int:
        # Step 1: Count how many times each character appears.
        # Counter gives us a dict-like {char: frequency}.
        freq = Counter(s)

        length = 0          # running length of the palindrome we can build
        has_odd = False     # tracks whether any character has an odd count

        # Step 2: Process each character's frequency.
        for count in freq.values():
            # Use as many pairs as possible.
            # If count is even -> use all of it.
```

```

# If count is odd -> use (count - 1), the largest even part.
# (count // 2) * 2 cleanly gives the largest even number <= count.
length += (count // 2) * 2

# If this character has an odd count, we have a leftover single.
if count % 2 == 1:
    has_odd = True

# Step 3: A single odd leftover can sit in the palindrome's center.
# We can only ever place ONE such center character.
if has_odd:
    length += 1

return length

```

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n)$	One pass to count characters (n = length of <code>s</code>), plus a pass over the frequency map (at most 52 entries — bounded constant).
Space	$O(1)$ (or $O(k)$)	The Counter stores at most 52 distinct keys (26 lowercase + 26 uppercase). Since this is bounded by the alphabet, it's effectively constant space.

5. Alternative Approaches

A. Set Toggle (clever, no explicit counts)

Toggle characters in/out of a set. After processing, the set holds only characters with **odd** counts. The number of "wasted" characters equals `len(set) - 1` (we keep one for the center).

```

def longestPalindrome(self, s: str) -> int:
    odd_chars = set()
    for c in s:
        if c in odd_chars:
            odd_chars.remove(c) # second occurrence -> becomes a pair
        else:
            odd_chars.add(c) # first occurrence -> currently odd
    # len(odd_chars) chars are unpaired; we can keep at most 1 of them.
    if odd_chars:
        return len(s) - len(odd_chars) + 1
    return len(s)

```

- **Insight:** `len(s) - (number of odd chars)` gives all pairable letters; adding `+1` reuses one odd as the center.
- Same $O(n)$ time, $O(1)$ space.

B. Count odd frequencies directly

Compute total length and subtract `(number_of_odd_counts - 1)` if there's any odd count. Same idea framed differently:

```

odd = sum(c % 2 for c in Counter(s).values())
return len(s) - odd + 1 if odd else len(s)

```

Key Takeaway

Pairing problems often reduce to **parity (even/odd) analysis** of frequencies. Here, every pair contributes 2, and a single odd leftover earns a `+1` bonus for the center — a recurring trick in palindrome construction problems.

Codex Solution

1. Intuition

A palindrome reads the same forward and backward.

For any character:

- We can place characters in **pairs** on both sides of the palindrome.
- Example: "cc" can be placed as one c on the left and one c on the right.
- If a character appears an **odd** number of times, we can use all except one of them as pairs.
- Example: "ccccc" has count 5, so we can use 4 of them as pairs.
- At most **one odd-count character** can be placed in the center of the palindrome.

So the key idea is:

Use all even counts, use `count - 1` from each odd count, and if there is at least one odd count, add 1 for the center.

2. Approach

Pseudocode

```
count frequency of each character in s
```

```
answer = 0
has_odd = false
```

```
for each character count:
    if count is even:
        answer += count
    else:
        answer += count - 1
        has_odd = true
```

```
if has_odd:
    answer += 1
```

```
return answer
```

3. Python Solution

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        # Dictionary to count how many times each character appears.
        # Since the problem is case-sensitive, 'A' and 'a' are counted separately.
        freq = {}

        for char in s:
            freq[char] = freq.get(char, 0) + 1

        # This will store the maximum palindrome length we can build.
        length = 0

        # Tracks whether we have at least one character with an odd frequency.
        # If yes, we can put exactly one such character in the center.
        has_odd = False

        for count in freq.values():
            if count % 2 == 0:
                # Even counts can be fully used.
                # Example: count = 4 -> use all 4 characters.
                length += count
            else:
                # Odd counts cannot be fully placed symmetrically.
                # We use count - 1 characters as pairs.
                # Example: count = 5 -> use 4 characters, leave 1 possible center.
                length += count - 1

                # Mark that we can place one odd character in the middle.
                has_odd = True

        # If any odd count existed, add one character as the center.
        if has_odd:
```

```
length += 1
return length
```

4. Complexity Analysis

Let n be the length of s .

Time Complexity: $O(n)$

We iterate through the string once to count frequencies.

Then we iterate through the character counts. Since the input only contains uppercase and lowercase English letters, there are at most 52 distinct characters.

So the total time is:

$$O(n + 52) = O(n)$$

Space Complexity: $O(1)$

There are only uppercase and lowercase English letters, so at most 52 possible keys in the frequency dictionary.

Therefore, the space usage is constant:

$$O(52) = O(1)$$

If we considered a general character set, the space complexity would be $O(k)$, where k is the number of distinct characters.

5. Alternative Approaches

Alternative 1: Using `collections.Counter`

This is a cleaner version of the same frequency-counting idea.

```
from collections import Counter

class Solution:
    def longestPalindrome(self, s: str) -> int:
        freq = Counter(s)

        length = 0
        has_odd = False

        for count in freq.values():
            length += count // 2 * 2

            if count % 2 == 1:
                has_odd = True

        return length + 1 if has_odd else length
```

Alternative 2: Using a Set to Track Unpaired Characters

We can track characters that currently have an odd count.

Whenever we see a character:

- If it is already in the set, remove it because we found a pair.
- Otherwise, add it because it is currently unpaired.

At the end:

- The set contains characters with odd counts.
- We can use all characters except one from each odd group.
- If there is at least one odd group, add one center character.

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        unpaired = set()

        for char in s:
```

```
    if char in unpaired:
        unpaired.remove(char)
    else:
        unpaired.add(char)

    # len(unpaired) is the number of characters with odd counts.
    # We must exclude one character from each odd group,
    # but can add one back as the center if any odd exists.
    if unpaired:
        return len(s) - len(unpaired) + 1

    return len(s)
```

This also runs in $O(n)$ time and $O(1)$ space because there are only 52 possible letters.